

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO5	Assessment:	ASSESSMENT TOOL2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)
Weightage:	30%	Total Marks:	25
CO5 Statement:	Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.		
Student Name:	Monal Raj.B	Reg.No.:	192511176
Department:	B.E -CSE	Date:	

ANNEXURE A
Pseudocode Writing Task (Algorithm Design)

1. Write pseudocode to design a DMA-based data transfer algorithm that moves data from an I/O device to memory while handling interrupts and minimizing CPU involvement.
2. Develop pseudocode for an interrupt handling system that prioritizes vectored interrupts from multiple devices and dispatches appropriate service routines.
3. Write pseudocode to select between memory-mapped I/O and I/O-mapped I/O dynamically based on latency and bandwidth requirements.
4. Design pseudocode for a bus arbitration algorithm that manages multiple devices communicating over PCIe, USB, and Thunderbolt interfaces.
5. Write pseudocode to implement a hybrid I/O communication mechanism that uses polling for low-speed devices and DMA with interrupts for high-speed devices.

Code of Conduct:

(192511176)
I Monal Raj.B (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO5	Assessment:	ASSESSMENT TOOL2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)
Weightage:	30%	Total Marks:	25
CO5 Statement:	<i>Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.</i>		
Student Name:	Monal Raj.B	Reg.No.:	192511176
Department:	B.E.CSE	Date:	

ANNEXURE A
Pseudocode Writing Task (Algorithm Design)

1. Write pseudocode to design a DMA-based data transfer algorithm that moves data from an I/O device to memory while handling interrupts and minimizing CPU involvement.
2. Develop pseudocode for an interrupt handling system that prioritizes vectored interrupts from multiple devices and dispatches appropriate service routines.
3. Write pseudocode to select between memory-mapped I/O and I/O-mapped I/O dynamically based on latency and bandwidth requirements.
4. Design pseudocode for a bus arbitration algorithm that manages multiple devices communicating over PCIe, USB, and Thunderbolt interfaces.
5. Write pseudocode to implement a hybrid I/O communication mechanism that uses polling for low-speed devices and DMA with interrupts for high-speed devices.

Answers

1. DMA-BASED DATA TRANSFER ALGORITHM

Aim

To design a DMA-based algorithm that transfers data from an I/O device to memory with minimal CPU involvement.

Pseudocode

BEGIN DMA_Data_Transfer

 INITIALIZE DMA_Controller

 INITIALIZE Memory_Buffer

 INITIALIZE Device

 CPU configures DMA:

 SET source_address $\leftarrow$ Device

 SET destination_address $\leftarrow$ Memory_Buffer

 SET data_size $\leftarrow$ N bytes

 ENABLE DMA_Controller

 WHILE transfer_not_complete DO

 DMA transfers data directly from Device to Memory

 END WHILE

 ON DMA_Completion_Interrupt:

 CPU executes ISR

 VERIFY data integrity

 SIGNAL transfer complete

END

Key Idea

- CPU only initializes transfer
- DMA handles bulk data movement
- Interrupt signals completion

2. VECTORED INTERRUPT HANDLING SYSTEM

Aim

To prioritize multiple interrupt sources and dispatch correct service routines.

Pseudocode

BEGIN Interrupt_Handler

 INITIALIZE Interrupt_Vector_Table

 SET priority_levels for devices

 WHILE system_running DO

 IF interrupt_detected THEN

 IDENTIFY interrupt_source

 FETCH ISR_address from vector_table

 IF higher_priority_interrupt THEN

 SAVE current_state

 EXECUTE high_priority_ISR

 RESTORE state

 ELSE

 EXECUTE ISR_address

 END IF

 END IF

END WHILE

END

Key Idea

- Vectored interrupts reduce lookup time
- Priority ensures critical tasks execute first

3. DYNAMIC I/O SELECTION ALGORITHM (MMIO vs I/O-MAPPED)

Aim

To dynamically select the best I/O method based on performance requirements.

Pseudocode

BEGIN IO_Selection

 INPUT latency_requirement

 INPUT bandwidth_requirement

 IF latency_requirement is HIGH THEN

 SELECT Memory_Mapped_IO

 ELSE IF bandwidth_requirement is HIGH THEN

 SELECT Memory_Mapped_IO

 ELSE

 SELECT IO_Mapped_IO

 END IF

 CONFIGURE system with selected_IO

END

Key Idea

- MMIO → faster access, high performance
- I/O-mapped → simpler, less address space usage

4. BUS ARBITRATION ALGORITHM

Aim

To manage fair communication among devices using PCIe, USB, and Thunderbolt.

Pseudocode

BEGIN Bus_Arbitration

 INITIALIZE device_queue

 INITIALIZE priority_levels

 WHILE bus_active DO

 FOR each device IN device_queue DO

 IF device_request = TRUE THEN

 APPLY arbitration_policy (Round Robin / Priority)

 IF bus_free THEN

 GRANT access to device

 device transmits data

 END IF

 END IF

 END FOR

END WHILE

END

Key Idea

- Ensures fairness
- Prevents starvation
- Supports multiple protocols

5. HYBRID I/O COMMUNICATION MECHANISM

Aim

To combine polling and DMA for efficient I/O handling.

Pseudocode

BEGIN Hybrid_IO_System

 INITIALIZE low_speed_devices

 INITIALIZE high_speed_devices

 WHILE system_running DO

 // Polling for low-speed devices

 FOR each device IN low_speed_devices DO

 CHECK device_status

 IF data_ready THEN

 READ data using CPU

 END IF

 END FOR

 // DMA for high-speed devices

 FOR each device IN high_speed_devices DO

IF large_data_transfer_required THEN

 CONFIGURE DMA:

 source $\leftarrow$ device

 destination $\leftarrow$ memory

 size $\leftarrow$ N

 START DMA transfer

END IF

END FOR

ON DMA_Completion:

 HANDLE interrupt

 CONFIRM transfer

END WHILE

END

Key Idea

- Polling $\rightarrow$ simple, good for slow devices
- DMA $\rightarrow$ efficient for large data
- Hybrid approach $\rightarrow$ optimized performance

Code of Conduct:

I Monal Raj.B (192511176) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *Monal Raj*

MARKS ALLOCATION

	Problem Understanding (20%)	Algorithm Design (30%)	Use of Control Structures (20%)	Pseudocode Structure (10%)	Completeness (20%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

MARKS ALLOCATION

	Problem Understanding (20%)	Algorithm Design (30%)	Use of Control Structures (20%)	Pseudocode Structure (10%)	Completeness (20%)
Q1	1	2.5	1	1	1.5
Q2	1.5	2	1.5	0.5	1.5
Q3	1.5	2	1.5	0.5	1
Q4	1	2.5	1	1	1
Q5	1.5	2	1	1	1.5

7
7
6.5
6.5
7

RUBRICS FOR CALCULATION:

32

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient